x: 1

t: 17/7/26

LINUX & WINDOWS COMMANDS

AIM: To study & execute various linux commands for File & directory management, text processing & basic networking operations.

→ Study of various commands used in Linux pwd

Description: prints the current working directory

Output:

```
pwd
/home/liveuser
mkdir
```

Description: creates a new directory

Output:

```
mkdir Fedora
ls
Desktop  Documents  downloads  Fedora
cd
```

Description: changes the current directory

Output:

```
$ cd Fedora
```

$pwd
/home/liveuser/.fedora
touch filename.txt.

Description: Creates an empty file
output:
$ touch example.txt
$ ls
example.txt
echo "Hi" > filename.txt.

Description: writes text into a file.
Output:
$ echo "Hi" > example.txt
cat filename.txt

Description: Displays the contents of the file
Output:
$ cat example.txt
Hi
grep

Description: Searches for a specific Pattern in a file

Output: $ grep "Hi" example.txt
Hi
wc

Description: Displays the no. of lines, words & bytes in a file.

Output:
$ wc example.txt
1 1 3 example.txt

WC-l

Description: counts the no. of. lines in a File.

Output:

```
$wc -l example.txt
  1 example.txt
  wc -w
```

Description: counts the no. of. words in a file.

Output:

```
$ wc -w example.txt
  1 example.txt
  wc -w
```

Description: counts the no. of. bytes in a File.

Output:

```
$wc -c example.txt
3 example.txt
wc -L
```

Description: counts the no. of characters in a File.

output:

```
$wc -m example.txt
  3 example.txt
  wc -L
```

Description: displays the length of the Longest line in a file

Output:

```
$wc -l example.txt
2 example.txt
```

Basic networking commands

hostname

Description: display the system is hostname

Output:

```
$hostname
localhost_line
```

route.

Description: display the routing table

Output:

```
$route
Kernel IP routing table
```

net state.

Description: shows network connections and routing information

Output:

```
$net state
Active Internet connection
```

ip address show

Description: displays IP address & network interface details

Output:

```
$ ip address show
1: lo:
2: enp0s3:
ns lookup
```

Description: finds the IP Address of a domain

Output:

```
$ns lookup www.google.com
name: www.google.com
Address: 142.250.xxx.xxx
if config
```

Description: Displays network int enforce configuration

Output:

```
int config
en pos 3
int 1°.0.2.15
Ping
```

Description: Tests connectivity to a host

Output: Ping www.google.com
64 bytes from...
amp

Description: displays the ARP cache

Output:

arp - 9

- gateway (10. 0. 2. 2)

traceroute

Description: displays the route taken by Packets to reach a destination

Output:

$ traceroute www. google. com

1 - gateway

2 . . .

Result: Thus the linux & windows commands are studied & executed Successfully.

Exp No: 2 Date: 23.7.26

Learning and Assignment of IP Address manually to computer

Aim: To Learn IP addressing & Manually assign are IP address to a computer To verify the assigned IP address using Linux networking commands

Theory:

An IP Address is a unique numerical address assigned to every device connected to a network. It enables communication blw devices over a network. An IPv4 address consist of four octlets seperated by dots (eg: 192.168.1.10)

An IP Address can be Assigned either Manually (static IP) or automatically (Dynamic IP) using DHCP. A manually assigned IP address remains unless changed by the User along with the IP address remains when parameters such as the Subnet Mask, default gateway & DNS server are also configured for proper network.

Commands & Sample Outputs:

1. Display all IP address

command:

ip address show

Sample-output:

2: ens160:
inet 172.16.211.128/24
inet6 Fc80::20c:29FF:Fe75:9927/64

Observation:

Displays the IPv_4 and IPv_6 address of all networks.

2. Display a specific network interface

command

ip addr show dev ens160

Sample-Output:

2: ens160
inet 172.16.211.128/24

3. Display Routing table

command

ip route show

Sample Output

default via 172.16.211.2 dev ens160
172.16.211.0/24 dev ens160

observation:- Displays the routing Table and the default gateway

4. Display Neighbour (ARP) Table :-

command

ip neigh show

sample output

172.16.211.1 REACHABLE
172.16.211.254 STABLE

observation:- Displays the neighbouring devices and their MAC address

Result:- The IP address was manually assigned and verified successfully using Linux networking commands.

Exp: 3 Date: 23.7.26

Study of different Types of Network cables & crimping of cable with RJ45 connector

Aim: To study different types of network cables and understand the crimping process using an RJ45 connector

Concept: Network cables are used to connect computers and offer networking devices for data communication the most commonly used cable in LAN is the unshielded Twisted pair (UTP) cable. An RJ45 connector is attached to the ends of the cable of establish a network connection. The wires are arranged according to the 568A or 568B wiring standard before crimping & the completed cable is tested to ensure proper connectivity.

Types of Network Cables:-

	cable type	Description
1.	UTP [Unshielded Twisted Pair]	commonly Used for Ethernet LAN's & office nchoorks
2.	STP [shielded Twisted Pair]	Used in area with High electrical or electromagnetic interference
3.	Coaxial cable	Used for cable television broadbarrel, internet & CCTV systems
4	optical Fiber cable	Used For High-speed High Bandwidth & Long distance communication.

Crimping Procedure:

1. cut the UTP cable to the required length
2. strip about 2-3 cm of the outer insulation
3. Untwist & straighten the wires
4. Arrange the wires according to the T568B standard
5. Trim the wire ends evenly
6. Insert the wires into the RJ45 connector.
7. crimp the connector Using a crimping tool
8. Test the cable using a LAN cable Tester

Observation:

* The wires were Arranged according to the T568B Standard
* The RJ45 connector was crimped Successfully
* The cable tester indicated proper continuity
* The cable was ready for network communication.

Result: The different types of network cables were studied, and the UTP cable was Successfully terminated using and RJ45 connector and Tested for proper connectivity

Exp: 4 Date:

Implement Packet Sniffing Using raw sockets in python.

Aim: To implement pocket sniffing using raw sockets in python & capture network packets passing through network interface

objective: To understand how raw sockets work & how network packets can be captured & analyze using python programming.

Theory:

1. Pocket Sniffing is the process of monitoring & capturing data packets travelling across a network

2. A pocket sniffer intercepts packets & analyzes their contents

3. Raw Sockets provide direct access to Lower-level network protocols, allowing applications to capture Packets

4. In python, the socket Module, can be used to create raw sockets for packet capture.

5. This Technique is useful for networking monitoring trouble shooting & security Analysis

Algorithm:

1. Import the required socket Module
2. Creat a raw socket
3. Bind the socket to the networks interface
4. Receive Packets using the recvfrom()
5. display the captured packet information
6. Repeat the process continuosly to monitor network traffic.

Program:

```
import socket:
    Sniffer = socket.Socket(socket.
    AF_INET, SOCKET, SOCK_RAW,
  SOCKET.IPROTIO_IP)
Sniffer.bind((host,0))
while True:
    raw-data, addr=Sniffer.recvFrom
                        (65535)
Print("Packet received From:", Addr)
Print(raw_data)
```

Output:-

Packet received from: ('192.168.15', 0)
b'E\x00\x00 c...\

Packet received from: ('192.168.1.10', 0)
b'E\x00\x00c ...

Result: Thus Packet Sniffing was successfully implemented Using raw sockets in python & network Packets were captured and displayed

Exp: 5 Date: 24/7/26

Develop a Customized Ping Command to test the Server connectivity

Aim: To test the Server connectivity and measure network performance using the ping command.

Uses OF Ping:

* Ping is a network diagnostic command used to check whether a server or Host is reachable over a network.

1. What does ping Measure?

Reachability - checks whether the destination server can be contacted

Latency - Measures the time taken for packets to travel to the server and back (in milliseconds)

Packet loss - shows whether any packets were lost during transmission, indicating Possible network problems.

1. When is ping Useful?

Ping is commonly Used to

* Verify that a Server is online
* Test network connectivity before Troubleshooting applications
* Measure network response time
* detect packet loss caused by unstable connections
* Perform basic network health checks

Procedure:

1. Basic ping command:

Ping www.google.com

Output: Displays IP address of the server, no of Packets sent, received, lost & the Avg round-trip time

2 Ping localHost Command:

Ping localhost

Output: Replies from 127.0.0.1 Confirm that the TCP/IP stack on local System is working Properly.

3. Ping Using IP Address command:

Ping 8.8.8.8

Output: Displays connectivity & delay time (in milliseconds) blw your system and Google DNS server

4. continuos ping Test command:

Ping www.google.com -t

Output: continuosly pings the server until stopped manually Showing live latency & packet information

5. ping with specific number of Echo requests command:

Ping www.google.com -n 5

Output: Sends exactly 5 echo request & displays the result with avg time & packet loss summary

6. ping with packet size Specification:

Ping www.google.com -l 1000

output: sends a ping request with Packet Size of 1000 bytes, showing transmission performance

Result: The ping commands were Successfully executed in windows command prompt & the network connectivity, latency & packet transmission blw local comp's & various servers were verified

Exp: 6

BUILDING ANONYMOUS FTP SCANNER USING FTPLIB MODULE

Aim: To create a python-based FTP Scanner using FTPLIB module

* check if a public FTP server allows anonymous Login
* lists contents of directories on server

Procedure:

* Import built-in Ftplib module
* write two python scripts:
  * Sender side: connects to FTP server
  * Receiver side: connects anonymously
* Run scripts in Google colab
* observe login status, server msg

code:

-> Sender

```
def anonymousLogin(hostname):
    try:
      ftp = ftplib.FTP(hostname)
          = ftp.login('anonymous', 'anonymous')
      Print (response)
      IF "230" in response:
    Print ("[+]" + str(hostname) + "FTP
      Anonymous Login Suceeded.")
    Print ("Welcome Message: \n", ftp.getwelcome())
    Print ("\n Directory listing:")
      ftp.dir()
      ftp.quit()
```

```
else:
    Print("[-]" + str(hostname) + "FTP Anonymous
    Login failed.")
except Exception as e: print("[-]" + str(hostname)
    + "FTP Anonymous Login failed.")
    Print("Error:", str(e))

'ftp.be.debian.org'
anonymous Login (hostname)
```

→ Receiver:

```
import ftplib
def check_anonymous_connection (host, path):
    try:
        with Ftplib.FTP (host) as connection:
            connection.login ('anonymous', 'anonymous')
        Print(F"\n listing directory: {path}")
        connection.cwd (path)
        for name, details in connection.mlsd():
        Print(name, details ['type'],
        details.get ('size', 'N/A'))
except Exception as e:
        Print("Error:", str(e))

FTP_SERVER_URL='ftp.be.debian.org'
DOWNLOAD_DIR_PATH='/pub/linux/kernel
check_anonymous_connection(FTP_server_URL,
    DOWNLOAD_DIR_PATH)
```

Output:

Sender side:

Ftp.be.debian.org FTP Anonymous Login Suceeded.

Welcome Message:

220 Pro FTPD Server (mirror.as35701.net)

Directory Listing:

drwxr-xr-x 9 ftp

Receiver side:

welcome: 220 Pro FTPD Server

listing directory: /pub/linux/kernel/v5.x/

linux-5.10.1.tar.x2 file 113072564 linux-5.10.2.tar.x2 file 113167444

Result:

* Script Successfully connected to FTP server (Ftp.be.debian.org)
* Verified anonymous login is Allowed
* directory contents & were retrieved & displayed
* This proves that the server provides Public Access & is working correctly

Exp: 7 Date:

Develop a Simple Calculator Using XMLRPC

Aim: To develop a simple calculator application Using XML-RPC Protocol in python that performs Basic arithmetic operations like addition, subraction, multiplication & division through client-server communication

Description: XML-RPC (extensible Markup Language Remote Procedure call) is a protocol that uses XML to encode requests & responses & HTTP as a transport mechanism. It enables communication between a client & a server where the client invokes methods hosted on the server remotely

* Server Holds arithmetic function
* The client connects to the server to perform operations like addition, subraction, multiplication & division

Code:

→ Server - Side code (server.py)

```
From Xmlrpc.server import
                    SimpleXLMRPC Server

def add (x,y):
    return x+y

def Subract (x,y):
    return x-y

def multiply (x,y):
    return x*y

def divide (x,y):
    if y!=0:
        return x/y
    else:
        return "Error: Division by zero"

Server = SimpleXMLRPC Server("localhost",
        8000, logRequests=False)

server.register_Function(add,"add")
server.register_Function(subract,"subract")
server.register_function(multiply,"multiply")
Server.register_Function(divide,"divide")
Print("XMLRPC Calculator Server is
        running on Port 8000...")
server.serve_forever()
```

```
def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

```
: 4
```

num1, num2

Print("Inputs: 30 and 10")

Print("Addition (+) :", proxy.add(num1, num2))

Print("Subraction (-) :", proxy.subract(num1, num2))

Print("Multiplication (*) :", Proxy.multiply(num1, num2))

Print("Division (/) :", proxy.divide(num1, num2))

Software Requirements:

* OS: Window / Linux
* python Version: 3.X or above
* Modules required: xmlrpc.server, xmlrpc.client (built-in with python)
* Text Editor or IDE: IDLE / VS CODE / Pycharm

PROCEDURE:

1. Open a python IDE or Terminal
2. create two seperate python files: server.py and client.py
3. write the provided server-side code in server.py & client-side code in client.py
4. Run the server.py file first to start XML-RPC server
5. Run the client.py to perform arithmetic operations remotely
6. Observe the results displayed on client screen & verify the server output

OUTPUT: Server side

XML-RPC server is running on Port 8000...

client-side:-

Inputs: 30 and 10
Addition : 40
Subraction : 20
Multiplication: 300
Division : 3.0

Result: The XML-RPC Based calculator was succesfully implemented. The client communicated with the server using XML-RPC, performed all arithmetic operations remotely and displayed accurate results.

Exp: 8

Simple chatting program in computer networks

Aim: To develop a simple chatting application using python socket Programming

Observation;

chatting allows users to exchange messages through a comp network socket programming provides communication blw two computers

The server waits for client connections, while the client converts to the serve. TCP ensures reliable data transmission

Procedure;

1. Install python & open VS code
2. create two files: server.py & client.py
3. import socket & threading Modules
4. create a TCP socket in both programs
5. Bind & start server, connect client
6. Run Server First & client next
7. exchange messages b/w two terminals
8. Type exit to end

code:

→Server

```
import socket
import threading
Server = socket.socket()
Server.bind(("127.0.0.1", 12345))
Server.listen(1)
Print("waiting for client...")
def receive():
    while True:
        msg = conn.req(1024).decode()
        if not msg or msg
            break
        Print("client:", msg)
threading.Thread(target=receive,
                 start())
while True:
    msg = input("server:")
    conn.sendall(msg.encode())
    if msg.lower() == "exit":
        break
Conn.close()
server.close()
```

client

```
import socket
import threading
client=socket.socket()
client=connect(("127.0.0.1", 12345))
Print("connected to server")

def receive():
    while True:
        msg=client.rec(1024).decode(
        if not msg or msg.lower()=="exi
            break
        Print("server:", msg)
threading.Thread(target=receive,
                 start())
while True:
    msg=input("client:")
    client.endall(msg.encode())
    if msg.lower()=="exit":
        break
client.close()
```

Server:

```
waiting for client
client connected
Server: Hello
client: Hi
Server: How are you?
Client: I am fine
```

client:
connected to server
client: Hi
Server: Hello
client: I am fine
Server: How are you?

Result: Thus, a simple chatting application was implemented successfully using python socket programming & TCP to exchange messages between client & server.

Exp:9 Date:

DESIGN A SIMPLE TOPOLOGY AND CONFIGURE WITH ONE ROUTER, TWO SWITCHES AND PCs Using CISCO PACKET TRACKER

Aim: To design and configure a simple network topology consisting of one router, two switches & multiple PC's in cisco packet tracer, and enable communication between PCs.

Definitions:

* Routers- A networking device that forwards data packets blw computer networks, directing traffic on internet

* switch = A device that connects devices within a network and uses a MAC addresses to forward data to the correct destination.

* Pc (personal computer): End devices in a network that users interact with to send and received data.

* cables:
(i) straight through cable: Used to connect diff devices like PC to switch or switch to Router.

[ii] Crossover cable: Used to connect similar devices like switch to switch [rarely needed with modern switches]

[iii] Console cable: Used to connect PC to router or switch console port for configuration.

Equipment Needed:-

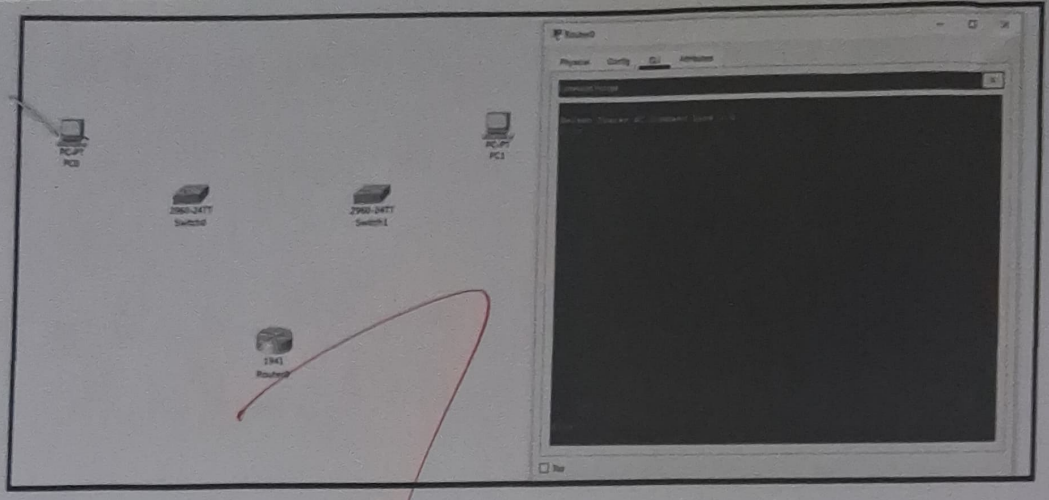
* Cisco packet Tracer software
* 1 Router (eg: 2811)
* 2 Switches (eg: 2960)
* 4 PC's
* Appropriate cables (copper straight-thro

Step-By-Step procedure:-

step 1: open cisco packet tracker
* Launch cisco packet tracker on your comp

Step 2: Add devices to workshop:-
* From the bottom device toolbar, drag & drop

drop :- * 1 Router (1941)
* 2 Switches (2960)
* 4 PC's (2 connected to each sw



fast ethernet 0/1.

PCs to the switches

*PC0 to switch 0 Fast Ethernet 0/2
*PC1 to switch 0 Fast Ethernet 0/3
* PC_2 to switch 1 Fast Ethernet 0/2
* PC_3 to switch 1 Fast Ethernet 0/3

Step 4: configure Router interface
*click the router
*click tab
* Enter the IP address & Give no shutdown command

Step 5: Configure Pc IP addresses
*click on Each Pc
*Desktop tab
*IP configuration & Assign IP addresses & Gateways

For Eg: PC0. 192.168.1.10
255.255.255.0

step 6: Save and Test connectivity

* Save the packet Tracer File
* Test the connectivity using the Ping command From PC0 to PC_2 or PC_3
* Go to CMD type ping: 192.168.2.10
* you should see replies if the network is connected successfully

Result: The PCs connected to different switches can communicate with each other via the router. The successful Ping replies verify that the network is properly configured & operational.

Exp:10 Date:

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACKER

Aim: To learn how to add a network module to a cisco switch, connect multiple Pc's & a server assign IP addresses & Test connectivity in cisco Packet tracker

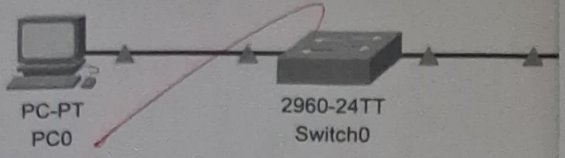
Definition: A network module is an extra component you can add to a switch. It gives you more ports or adds new types of connections in packet tracer, modules such as HWIC-4ESW let you expand the switch's capabilities

Procedure:

1. open cisco packet tracker & set up your workspace.
2. drag and drop a 2960-24TT switch to the center.
3. add five Pc's and one server PT from end devices panel.
4. power off the switch:
   - click the switch, open physical lab, & turn off the power button

on the

IWIC-4ES



boot

opper

Straight Through cable.

PC0 - switch Fast Ethernet 0/1

PC1 - switch Fast Ethernet 0/2

PC2 - switch Module Port 1

PC3 - switch Module Port 2

PC4 - switch Module port 3.

Server 0 -> switch Module port 4 8.

Assign IP addresses to each device:

PC0: 192.168.1.10

PC1: 192.168.1.11

PC2: 192.168.1.12

PC3: 192.168.1.13 - PC4: 192.168.1.14

Server 0: 192.168.1.100

All Subnet masks: 255.255.255.0

8 Test Connectivity (From any Pc - open command prompt & ping other devices)

Input:

-> Devices:

1 Cisco 2960-24TT switch

1 HWIC-4ESW network module

5 PCs (PC0 - PC4)

1 Server-PT (Server 0)

cables:-

* copper straight-through cables IP configuration

* use static IP addresses as listed above

Output: A working network setup where every PC & then the server can successfully ping each other, proving that both built-in & module ports on the switch are functioning.

Result: The experiment demonstrates successfully customization of a switch with a network module & connectivity b/w all devices. The networks works as expected & all devices communicate.

Exp: 11

Examine network Address Translation Using cisco packet tracker

Aim: Implement network address Translation using cisco packet tracker we'll configure static, dynamic & port addresses Translation on a router

Procedure:

Step 1: Setup network Topology launch cisco packet tracker software

Step 2: add devices

Step 3: connect devices

Step 4: Verify connections

Step 5: Assign IP addresses

Step 6: Verify IP co

code:

```
#Enable configure Terminal
interface Gigabit Ethernet 0/0 i|p
 address 192.168.1.1 255.255.255.0
```

4.1 configure Inside & outside Interfaces for NAT

```
configure terminal interface
Gigabit Ethernet 0/0 ip hat
inside exit Interface
Gigabit Ethernet 0/1 ip hat
Outside exit
```

4.2 Static NAT Configuration (Example)

inside source static 192.168.1.10 200.1.1.10

ip nat pool My.pool 200.1.1.2 0 200.1.1.25
natmask 255.255.255.0

access_list 1 permit 192.168. 1.0.0.00.255

ipnat inside source list (pool My.pool)

TO SET DEFAULT GATEWAY:

1. click on PC1 in the workspace
2. Goto the desktop tab
3. click on IP configuration
4. In the default gateway Fieldstype
   192.168.1.1
5. make sure the IP address & subnet mask are already set
6. close the window.

Result: The Router succesfully translated Private IPs to public one Using static, dynamic & PAT configurations.

Σ :12

NMAP TO DISCOVER LIVE HOSTS USING ARP, ICMP AND TCP/UDP PING SCAN IN TRYHACKME

Aim: Learn & Implement Nmap host discovery techniques (ARP, ICMP variants, TCP, UDP pings) to identify live hosts on a network & compare their effectiveness

Description:
Use Nmap probe types to discover which IPs are active before port/vulnerability scans
Covers ARP, ICMP echo, TCP Syn, UDP Probes & combined scans

Definition:
Host discovery: Finding live systems
PR(ARP): Layer 2 MAC resolution
PE (ICMP Echo): Standard ping
PP (ICMP Timestamp) / -PM (ICMP (Mask)
PS (TCP syn): send Syn to specified ports
PA (TCP Ack): send ACK to specified ports
PU (UDP): send UDP to ports

Procedure:
ARP (LAN) nmap -sn -PR 10.10.X.0/24
nmap -sn -PR 192.168.1.0/24 -packet trace

ICMP Echo/Timestamp/Mask nmap -sn -PE 10.10.X.0/24 nmap -sn -PP 10.10.X.100
nmap -sn -pM 10.10.X.100

TCP syn/ACK (specify ports)
-PE 10.10.X.0/24 nmap -sn -PP 10.10.X.100
nmap -sn -PA 80, 443, 10.10.X.0/24

UDP nmap -sn -PU53,161 10.10.X.0/24
combined (disable ARP nmap -sn -PE
-PS -PA -PU 10.10.X.0/24 nmap -sn
10.10.X.0/24 --disable-arp-ping INPUT
Target IP or range (eg: 10.10.X.100 or
10.10.X.0/24)

Optional port lists for -PS, -PA, -PU
(eg: 22,80,443,53,161

Flags:- -sn, -PR, -PE, -PP, -PM, -PS,
-PA, -PU, - Packet-trace,
--disable-arp-ping

Output:
ARP
Nmap scan report for 10.10.X.1 Host
is up (0.00050s latency)
MAC address: 02:83:75:3A:F2:89
Nmap done: 256 IP addresses (3 hosts up)
ICMP
Nmap scan report for 10.10.X.100 Host
is up (0.00099s latency)
Nmap done: 1 IP address (1 host up)

Result: ARP discovery reliably identified hosts on local subnet with lowest latency & Highest Accuracy while ICMP, TCP (Syn/Ack) and UDP Probes found hosts

Ex:13

DEMONSTRATE NETWORK FORENSICS USING PCAPXRAY TOOL

Aim: To analyze & visualize network traffic from a PCAP file using PcapXray tool for forensic investigation

Steps to complete

St-1: Prepare your computer

Make sure you have python 3 installed
IF u don't Have:
*Go to python.org & download it

St-2: Install PcapXray tool

1. open command prompt
2. Type this commend & Enter:
3. Pip install pcapxray.

St-3: Get a PCAP file

Option A: Use a sample PCAP file
download PCAP file from website like
https://wiki.wireshark.org/SampleCaptures
& Save

Option B: Capture your own traffic

St-4: Run PcapXray on your PCAP file

1. Put the PCAP file in folder
2. open command prompt
3. cd path_to_your_folder
4. Run this command
5. PcapXray -f capture.pcap

St. 5: View the Results
after command Runs, create a file named
PcapXray.html
* protocols & ports Used

st. 6: Analyze
* Look for biggest nodes
* notice Unusual ports or repeated connection

Result: PcapXray Successfully Generated a Visual maps of hosts, connections & Protocols.

EX: 14 TO CAPTURE SAVE AND ANALYZE NETWORK TRAFFIC ON TCP/UDP/IP/HTTP/ ARP/DHCP/ICMP/DNS USING WIRESHARK TOOL

Aim: The aim of this experiment is to capture, save & analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/ DNS Using wireshark tool.

Procedure:

Step 1: download & install wireshark from its official website.

Step 2: Launch wireshark & identify the network interface

Step 3: Before starting the capture, apply capture filters to focus on specific protocols

Capture only TCP: tcp, capture only UDP: udp
Capture only HTTP: tcp port 80, Capture only DNS: udp Port 53
capture only ICMP: icmp, capture only ARP: arp
capture DHCP: port 67 or 68

Step 4: click start to begin capturing live network traffic.

Step 5: capturing, use display filters to isolate & analyze specific protocols

Step 6: let capture run until you have collected enough packets for Analysis

Step 7: save captured traffic by navigating

Step 8: Review & analyze captured packets in detail, Look for Source & destination IP addresses, protocol types, packet lengths & Flags

Output:
No: packet number in capture sequence
Time: packet was captured
Source, destination
Protocol: TCP, UDP, HTTP, ARP, DNS
Length: size of packets in bytes

sample-output:

1. 0.00000 192.168.1.1 8.8.8.8 ICMP 56
Echo request

2. 0.002345 8.8.8.8 192.168.1.1 ICMP
64 Echo Reply

3. 0.003234 192.168.1.2 192.168.1.1
ARP 42 who has 192.168.1.1?

Result: Thus this experiment capture, sa
& analyze network traffic on TCP/UDP/(A
HTTP/ARP/DHCP/ICMP/DNS using
wireshark tool

Exp: 15 ANALYZE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

Aim: To analyze usage statistics of diff types of servers using webalizer tool

Procedure:

1. Install webalizer tool in the system
   sudo apt-get update sudo
2. verify installation
   webalizer -V
3. Locate Log files of servers:
   Apache: /var/log/apache2/access.log
   Nginx: /var/log/nginx/access.log
4. Run webalizer on log file to Generate a report.
5. Open a web browser & access report
6. repeat same process for Nginx, FTP or Proxy logs
7. compare & analyze reports

Program:

step 1: Install webalizer sudo apt-get install webalizer for apache logs

step 2: Run webalizer for Apache Logs

step 3: Run webalizer for Nginx Logs

step 4: Run webalizer for proxy logs sudo webalizer

Output:

Server Type
Total Hits
Unique Visitors
Bandwidth Used

Top
Client / Domain Report URL
Apache: 12,560 1,240 1.8 GB
Nginx 9,840 980 1.2 GB
Proxy 15,300 1,450 2.1 GB
Graphical output
Monthly summary chart
Daily / Hourly usage histogram
Pie-chart of top Hosts & URLS

Result: webalizer tool successfully Generated graphical & stastical reports of diff servers
analysis provided insights into
Total Hits & visits
Bandwidth Usage
Top IPS & URLS
daily, monthly & hourly traffic.